Wav2Lip Documentation

1. Import basic modules

```python
import os
from os.path import dirname, join, basename, isfile
```

2. Create directories

```python
os.mkdir('temp')
os.mkdir('result')
```

3. Set syncnet_T parameter

```python
syncnet_T = 5
```

4. Clone Wav2Lip repository

```python
!git clone https://github.com/Rudrabha/Wav2Lip.git
```

5. Install requirements

```python
!pip install -r /kaggle/working/Wav2Lip/requirements.txt
```

6. Install librosa

```python
!pip install librosa==0.8.1
```

7. Import additional modules

```python
import os, subprocess
```

8. Get video names

```python
video_names=os.listdir("/kaggle/input/ur-dataset/ur-dataset/data")
```

9. Change directory

```python
%cd /kaggle/working/Wav2Lip
```

10. Download pretrained models

```python
!wget 'https://iiitaphyd-my.sharepoint.com/personal/radrabha_m_research_iiit_ac_in/_layouts/15/download.aspx?share=EdjI7bZlgApMqsVoEUUXpLsBxqXbn5z8VTmoxp55YNDcIA' -O '/kaggle/working/Wav2Lip/checkpoints/wav2lip_gan.pth'

!wget 'https://iiitaphyd-my.sharepoint.com/:u:/g/personal/radrabha_m_research_iiit_ac_in/Eb3LEzbfuKlJiR600lQWRxgBIY27JZg80f7V9jtMfbNDaQ?e=TBFBVW' -O '/kaggle/working/Wav2Lip/checkpoints/wav2lip.pth'

!pip install https://raw.githubusercontent.com/AwaleSajil/ghc/master/ghc-1.0-py3-none-any.whl

!wget "https://www.adrianbulat.com/downloads/python-fan/s3fd-619a316812.pth" -O "/kaggle/working/Wav2Lip/face_detection/detection/sfd/s3fd.pth"
```

11. Create train.txt

```python
with open("/kaggle/working/Wav2Lip/filelists/train.txt","w") as file:
  for i in os.listdir("/kaggle/input/ur-dataset/ur-dataset/data")[20:]:
      file.write("/kaggle/input/ur-dataset/ur-dataset/data/"+i+"\n")
```

12. Create val.txt

```python
with open("/kaggle/working/Wav2Lip/filelists/val.txt","w") as file:
  for i in os.listdir("/kaggle/input/ur-dataset/ur-dataset/data")[0:10]:
      file.write("/kaggle/input/ur-dataset/ur-dataset/data/"+i+"\n")
```

13. Create test.txt

```python
with open("/kaggle/working/Wav2Lip/filelists/test.txt","w") as file:
  for i in os.listdir("/kaggle/input/ur-dataset/ur-dataset/data")[10:20]:
      file.write("/kaggle/input/ur-dataset/ur-dataset/data/"+i+"\n")
```

14. Conv2d class definition

```python
import torch
from torch import nn
from torch.nn import functional as F

class Conv2d(nn.Module):
  def __init__(self, cin, cout, kernel_size, stride, padding, residual=False, *args, **kwargs):
    super().__init__(*args, **kwargs)
```

```
        self.conv_block = nn.Sequential(
                            nn.Conv2d(cin, cout, kernel_size, stride, padding),
                            nn.BatchNorm2d(cout)
                            )
        self.act = nn.ReLU()
        self.residual = residual

    def forward(self, x):
        out = self.conv_block(x)
        if self.residual:
            out += x
        return self.act(out)
```

15. SyncNet_color class definition

```python
import torch
from torch import nn
from torch.nn import functional as F

class SyncNet_color(nn.Module):
    def __init__(self):
        super(SyncNet_color, self).__init__()
```

```
self.face_encoder = nn.Sequential(
    Conv2d(15, 32, kernel_size=(7, 7), stride=1, padding=3),
    Conv2d(32, 64, kernel_size=5, stride=(1, 2), padding=1),
    Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
    Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(128, 256, kernel_size=3, stride=2, padding=1),
    Conv2d(256, 256, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(256, 256, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(256, 512, kernel_size=3, stride=2, padding=1),
    Conv2d(512, 512, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(512, 512, kernel_size=3, stride=1, padding=1, residual=True),
    Conv2d(512, 512, kernel_size=3, stride=2, padding=1),
    Conv2d(512, 512, kernel_size=3, stride=1, padding=0),
```

```
        Conv2d(512, 512, kernel_size=1, stride=1, padding=0),)


    self.audio_encoder = nn.Sequential(
        Conv2d(1, 32, kernel_size=3, stride=1, padding=1),
        Conv2d(32, 32, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(32, 32, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(32, 64, kernel_size=3, stride=(3, 1), padding=1),
        Conv2d(64, 64, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(64, 64, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(64, 128, kernel_size=3, stride=3, padding=1),
        Conv2d(128, 128, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(128, 128, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(128, 256, kernel_size=3, stride=(3, 2), padding=1),
        Conv2d(256, 256, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(256, 256, kernel_size=3, stride=1, padding=1,
residual=True),
        Conv2d(256, 512, kernel_size=3, stride=1, padding=0),
        Conv2d(512, 512, kernel_size=1, stride=1, padding=0),)
```

```
    def forward(self, audio_sequences, face_sequences):
        face_embedding = self.face_encoder(face_sequences)
        audio_embedding = self.audio_encoder(audio_sequences)
        audio_embedding =
audio_embedding.view(audio_embedding.size(0), -1)
        face_embedding =
face_embedding.view(face_embedding.size(0), -1)
        audio_embedding = F.normalize(audio_embedding, p=2, dim=1)
        face_embedding = F.normalize(face_embedding, p=2, dim=1)
        return audio_embedding, face_embedding
```

16. Import additional modules

```python
from os.path import dirname, join, basename, isfile
from tqdm import tqdm
from models import SyncNet_color as SyncNet
import audio
import librosa
import torch
from torch import nn
from torch import optim
import torch.backends.cudnn as cudnn
from torch.utils import data as data_utils
```

```
import numpy as np
from glob import glob
import os, random, cv2, argparse
from hparams import hparams, get_image_list
```

17. Initialize training parameters

```python
global_step = 0
global_epoch = 0
use_cuda = torch.cuda.is_available()
print('use_cuda: {}'.format(use_cuda))
syncnet_T = 5
syncnet_mel_step_size = 16
data_root="/kaggle/input/ur-dataset/ur-dataset/data"
```

18. Dataset class definition

```python
class Dataset(object):
    def __init__(self, split):
        self.all_videos = get_image_list(data_root, split)

    def get_frame_id(self, frame):
```

```
    return int(basename(frame).split('.')[0])

def get_window(self, start_frame):
    start_id = self.get_frame_id(start_frame)
    vidname = dirname(start_frame)
    window_fnames = []
    for frame_id in range(start_id, start_id + syncnet_T):
        frame = join(vidname, '{}.jpg'.format(frame_id))
        if not isfile(frame):
            return None
        window_fnames.append(frame)
    return window_fnames

def crop_audio_window(self, spec, start_frame):
    start_frame_num = self.get_frame_id(start_frame)
    start_idx = int(80. * (start_frame_num / float(hparams.fps)))
    end_idx = start_idx + syncnet_mel_step_size
    return spec[start_idx : end_idx, :]

def __len__(self):
    return len(self.all_videos)

def __getitem__(self, idx):
    while 1:
```

```
idx = random.randint(0, len(self.all_videos) - 1)
vidname = self.all_videos[idx]
img_names = list(glob(join(vidname, '*.jpg')))
if len(img_names) <= 3 * syncnet_T:
    continue
img_name = random.choice(img_names)
wrong_img_name = random.choice(img_names)
while wrong_img_name == img_name:
    wrong_img_name = random.choice(img_names)

if random.choice([True, False]):
    y = torch.ones(1).float()
    chosen = img_name
else:
    y = torch.zeros(1).float()
    chosen = wrong_img_name

window_fnames = self.get_window(chosen)
if window_fnames is None:
    continue

window = []
all_read = True
for fname in window_fnames:
```

```
            img = cv2.imread(fname)
            if img is None:
                all_read = False
                break
            try:
                img = cv2.resize(img, (hparams.img_size,
hparams.img_size))
            except Exception as e:
                all_read = False
                break
            window.append(img)

        if not all_read: continue

        try:
            wavpath = join(vidname, "audio.wav")
            wav = audio.load_wav(wavpath, hparams.sample_rate)
            orig_mel = audio.melspectrogram(wav).T
        except Exception as e:
            print(e)
            continue

        mel = self.crop_audio_window(orig_mel.copy(), img_name)
        if (mel.shape[0] != syncnet_mel_step_size):
```

```
            continue

        x = np.concatenate(window, axis=2) / 255.
        x = x.transpose(2, 0, 1)
        x = x[:, x.shape[1]//2:]
        x = torch.FloatTensor(x)
        mel = torch.FloatTensor(mel.T).unsqueeze(0)
        return x, mel, y
```

19. Test Dataset class

```python
ds=Dataset("train")
x,mel,t=ds[0]
print(x.shape)
print(mel.shape)
print(t.shape)
```

20. Wav2Lip model definition

```python
class nonorm_Conv2d(nn.Module):
    def __init__(self, cin, cout, kernel_size, stride, padding,
residual=False, *args, **kwargs):
```

```
        super().__init__(*args, **kwargs)
        self.conv_block = nn.Sequential(
                        nn.Conv2d(cin, cout, kernel_size, stride, padding),
                        )
        self.act = nn.LeakyReLU(0.01, inplace=True)

    def forward(self, x):
        out = self.conv_block(x)
        return self.act(out)

class Conv2dTranspose(nn.Module):
    def __init__(self, cin, cout, kernel_size, stride, padding,
output_padding=0, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.conv_block = nn.Sequential(
                        nn.ConvTranspose2d(cin, cout, kernel_size, stride,
padding, output_padding),
                        nn.BatchNorm2d(cout)
                        )
        self.act = nn.ReLU()

    def forward(self, x):
        out = self.conv_block(x)
        return self.act(out)
```

```
class Wav2Lip(nn.Module):
   def __init__(self):
      super(Wav2Lip, self).__init__()
      self.face_encoder_blocks = nn.ModuleList([
         nn.Sequential(Conv2d(6, 16, kernel_size=7, stride=1, padding=3)),
         nn.Sequential(Conv2d(16, 32, kernel_size=3, stride=2, padding=1),
         Conv2d(32, 32, kernel_size=3, stride=1, padding=1, residual=True),
         Conv2d(32, 32, kernel_size=3, stride=1, padding=1, residual=True)),
         nn.Sequential(Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
         Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True),
         Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True),
         Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True)),
         nn.Sequential(Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
         Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True),
         Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True)),
```

```
        nn.Sequential(Conv2d(128, 256, kernel_size=3, stride=2,
padding=1),

        Conv2d(256, 256, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(256, 256, kernel_size=3, stride=1, padding=1,
residual=True)),

        nn.Sequential(Conv2d(256, 512, kernel_size=3, stride=2,
padding=1),

        Conv2d(512, 512, kernel_size=3, stride=1, padding=1,
residual=True),),

        nn.Sequential(Conv2d(512, 512, kernel_size=3, stride=1,
padding=0),

        Conv2d(512, 512, kernel_size=1, stride=1, padding=0)),])


    self.audio_encoder = nn.Sequential(

        Conv2d(1, 32, kernel_size=3, stride=1, padding=1),

        Conv2d(32, 32, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(32, 32, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(32, 64, kernel_size=3, stride=(3, 1), padding=1),

        Conv2d(64, 64, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(64, 64, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(64, 128, kernel_size=3, stride=3, padding=1),
```

```
        Conv2d(128, 128, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(128, 128, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(128, 256, kernel_size=3, stride=(3, 2), padding=1),

        Conv2d(256, 256, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(256, 512, kernel_size=3, stride=1, padding=0),

        Conv2d(512, 512, kernel_size=1, stride=1, padding=0),)


      self.face_decoder_blocks = nn.ModuleList([

        nn.Sequential(Conv2d(512, 512, kernel_size=1, stride=1,
padding=0),),

        nn.Sequential(Conv2dTranspose(1024, 512, kernel_size=3,
stride=1, padding=0),

        Conv2d(512, 512, kernel_size=3, stride=1, padding=1,
residual=True),),

        nn.Sequential(Conv2dTranspose(1024, 512, kernel_size=3,
stride=2, padding=1, output_padding=1),

        Conv2d(512, 512, kernel_size=3, stride=1, padding=1,
residual=True),

        Conv2d(512, 512, kernel_size=3, stride=1, padding=1,
residual=True),),

        nn.Sequential(Conv2dTranspose(768, 384, kernel_size=3,
stride=2, padding=1, output_padding=1),

        Conv2d(384, 384, kernel_size=3, stride=1, padding=1,
residual=True),
```

```
        Conv2d(384, 384, kernel_size=3, stride=1, padding=1, residual=True),),

        nn.Sequential(Conv2dTranspose(512, 256, kernel_size=3, stride=2, padding=1, output_padding=1),

        Conv2d(256, 256, kernel_size=3, stride=1, padding=1, residual=True),

        Conv2d(256, 256, kernel_size=3, stride=1, padding=1, residual=True),),

        nn.Sequential(Conv2dTranspose(320, 128, kernel_size=3, stride=2, padding=1, output_padding=1),

        Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True),

        Conv2d(128, 128, kernel_size=3, stride=1, padding=1, residual=True),),

        nn.Sequential(Conv2dTranspose(160, 64, kernel_size=3, stride=2, padding=1, output_padding=1),

        Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True),

        Conv2d(64, 64, kernel_size=3, stride=1, padding=1, residual=True),)])


      self.output_block = nn.Sequential(Conv2d(80, 32, kernel_size=3, stride=1, padding=1),

        nn.Conv2d(32, 3, kernel_size=1, stride=1, padding=0),

        nn.Sigmoid())


   def forward(self, audio_sequences, face_sequences):
```

```
        B = audio_sequences.size(0)
        input_dim_size = len(face_sequences.size())
        if input_dim_size > 4:
            audio_sequences = torch.cat([audio_sequences[:, i] for i in
range(audio_sequences.size(1))], dim=0)
            face_sequences = torch.cat([face_sequences[:, :, i] for i in
range(face_sequences.size(2))], dim=0)

        audio_embedding = self.audio_encoder(audio_sequences)
        feats = []
        x = face_sequences
        for f in self.face_encoder_blocks:
            x = f(x)
            feats.append(x)

        x = audio_embedding
        for f in self.face_decoder_blocks:
            x = f(x)
            try:
                x = torch.cat((x, feats[-1]), dim=1)
            except Exception as e:
                print(x.size())
                print(feats[-1].size())
                raise e
```

```
            feats.pop()

        x = self.output_block(x)
        if input_dim_size > 4:
            x = torch.split(x, B, dim=0)
            outputs = torch.stack(x, dim=2)
        else:
            outputs = x
        return outputs
```

21. Wav2Lip discriminator definition

```python
class Wav2Lip_disc_qual(nn.Module):
    def __init__(self):
        super(Wav2Lip_disc_qual, self).__init__()
        self.face_encoder_blocks = nn.ModuleList([
            nn.Sequential(nonorm_Conv2d(3, 32, kernel_size=7, stride=1, padding=3)),
            nn.Sequential(nonorm_Conv2d(32, 64, kernel_size=5, stride=(1, 2), padding=2),
            nonorm_Conv2d(64, 64, kernel_size=5, stride=1, padding=2)),
            nn.Sequential(nonorm_Conv2d(64, 128, kernel_size=5, stride=2, padding=2),
```

```
            nonorm_Conv2d(128, 128, kernel_size=5, stride=1,
padding=2)),
            nn.Sequential(nonorm_Conv2d(128, 256, kernel_size=5,
stride=2, padding=2),
            nonorm_Conv2d(256, 256, kernel_size=5, stride=1,
padding=2)),
            nn.Sequential(nonorm_Conv2d(256, 512, kernel_size=3,
stride=2, padding=1),
            nonorm_Conv2d(512, 512, kernel_size=3, stride=1,
padding=1)),
            nn.Sequential(nonorm_Conv2d(512, 512, kernel_size=3,
stride=2, padding=1),
            nonorm_Conv2d(512, 512, kernel_size=3, stride=1,
padding=1),),
            nn.Sequential(nonorm_Conv2d(512, 512, kernel_size=3,
stride=1, padding=0),
            nonorm_Conv2d(512, 512, kernel_size=1, stride=1,
padding=0)),])
        self.binary_pred = nn.Sequential(nn.Conv2d(512, 1,
kernel_size=1, stride=1, padding=0), nn.Sigmoid())
        self.label_noise = .0

    def get_lower_half(self, face_sequences):
        return face_sequences[:, :, face_sequences.size(2)//2:]

    def to_2d(self, face_sequences):
        B = face_sequences.size(0)
```

```
        face_sequences = torch.cat([face_sequences[:, :, i] for i in
range(face_sequences.size(2))], dim=0)
        return face_sequences


    def perceptual_forward(self, false_face_sequences):
        false_face_sequences = self.to_2d(false_face_sequences)
        false_face_sequences =
self.get_lower_half(false_face_sequences)
        false_feats = false_face_sequences
        for f in self.face_encoder_blocks:
            false_feats = f(false_feats)
        false_pred_loss =
F.binary_cross_entropy(self.binary_pred(false_feats).view(len(false_
feats), -1),
                               torch.ones((len(false_feats), 1)).cuda())
        return false_pred_loss


    def forward(self, face_sequences):
        face_sequences = self.to_2d(face_sequences)
        face_sequences = self.get_lower_half(face_sequences)
        x = face_sequences
        for f in self.face_encoder_blocks:
            x = f(x)
        return self.binary_pred(x).view(len(x), -1)
```

22. Training and evaluation functions

```python
logloss = nn.BCELoss()
def cosine_loss(a, v, y):
    d = nn.functional.cosine_similarity(a, v)
    loss = logloss(d.unsqueeze(1), y)
    return loss


def train(device, model, train_data_loader, test_data_loader, optimizer,
          checkpoint_dir=None, checkpoint_interval=None, nepochs=None):
    global global_step, global_epoch
    resumed_step = global_step

    while nepochs >0:
        nepochs-=1
        print('Starting Epoch: {}'.format(global_epoch))
        running_sync_loss, running_l1_loss = 0., 0.
        prog_bar = tqdm(enumerate(train_data_loader))
        for step, (x, indiv_mels, mel, gt) in prog_bar:
            model.train()
            optimizer.zero_grad()
```

```
        x = x.to(device)
        mel = mel.to(device)
        indiv_mels = indiv_mels.to(device)
        gt = gt.to(device)
        g = model(indiv_mels, x)
        if hparams.syncnet_wt > 0.:
            sync_loss = get_sync_loss(mel, g)
        else:
            sync_loss = 0.
        l1loss = recon_loss(g, gt)
        loss = hparams.syncnet_wt * sync_loss + (1 -
hparams.syncnet_wt) * l1loss
        loss.backward()
        optimizer.step()
        global_step += 1
        cur_session_steps = global_step - resumed_step
        running_l1_loss += l1loss.item()
        if hparams.syncnet_wt > 0.:
            running_sync_loss += sync_loss.item()
        else:
            running_sync_loss += 0.
        if global_step == 1 or global_step % checkpoint_interval == 0:
            save_checkpoint(
```

```
                model, optimizer, global_step, checkpoint_dir,
global_epoch)

            if global_step == 1 or global_step % hparams.eval_interval ==
0:

                with torch.no_grad():

                    average_sync_loss = eval_model(test_data_loader,
global_step, device, model, checkpoint_dir)

                    if average_sync_loss < .75:

                        hparams.set_hparam('syncnet_wt', 0.01)

            prog_bar.set_description('L1: {}, Sync Loss:
{}'.format(running_l1_loss / (step + 1),

                                                    running_sync_loss / (step + 1)))

        global_epoch += 1


def eval_model(test_data_loader, global_step, device, model,
checkpoint_dir):

    eval_steps = 700

    print('Evaluating for {} steps'.format(eval_steps))

    sync_losses, recon_losses = [], []

    step = 0

    while 1:

        for x, indiv_mels, mel, gt in test_data_loader:

            step += 1

            model.eval()

            x = x.to(device)
```

```
            gt = gt.to(device)
            indiv_mels = indiv_mels.to(device)
            mel = mel.to(device)
            g = model(indiv_mels, x)
            sync_loss = get_sync_loss(mel, g)
            l1loss = recon_loss(g, gt)
            sync_losses.append(sync_loss.item())
            recon_losses.append(l1loss.item())
            if step > eval_steps:
                averaged_sync_loss = sum(sync_losses) / len(sync_losses)
                averaged_recon_loss = sum(recon_losses) /
len(recon_losses)
                print('L1: {}, Sync loss: {}'.format(averaged_recon_loss,
averaged_sync_loss))
                return averaged_sync_loss


def save_checkpoint(model, optimizer, step, checkpoint_dir, epoch):
    checkpoint_path = join(
        checkpoint_dir,
"checkpoint_step{:09d}.pth".format(global_step))
    optimizer_state = optimizer.state_dict() if
hparams.save_optimizer_state else None
    torch.save({
        "state_dict": model.state_dict(),
        "optimizer": optimizer_state,
```

```
        "global_step": step,
        "global_epoch": epoch,
    }, checkpoint_path)
    print("Saved checkpoint:", checkpoint_path)


def _load(checkpoint_path):
    if use_cuda:
        checkpoint = torch.load(checkpoint_path)
    else:
        checkpoint = torch.load(checkpoint_path,
                                map_location=lambda storage, loc: storage)
    return checkpoint


def load_checkpoint(path, model, optimizer, reset_optimizer=False,
overwrite_global_states=True):
    global global_step
    global global_epoch
    print("Load checkpoint from: {}".format(path))
    checkpoint = _load(path)
    s = checkpoint["state_dict"]
    new_s = {}
    for k, v in s.items():
        new_s[k.replace('module.', '')] = v
    model.load_state_dict(new_s)
```

```
    if not reset_optimizer:
        optimizer_state = checkpoint["optimizer"]
        if optimizer_state is not None:
            print("Load optimizer state from {}".format(path))
            optimizer.load_state_dict(checkpoint["optimizer"])
    if overwrite_global_states:
        global_step = checkpoint["global_step"]
        global_epoch = checkpoint["global_epoch"]
    return model
```

23. Initialize training

```python
train_dataset = Dataset('train')
test_dataset = Dataset('val')
train_data_loader = data_utils.DataLoader(
    train_dataset, batch_size=hparams.batch_size, shuffle=True,
    num_workers=2)
test_data_loader = data_utils.DataLoader(
    test_dataset, batch_size=hparams.batch_size,
    num_workers=2)
device = torch.device("cuda" if use_cuda else "cpu")
model = Wav2Lip().to(device)
```

```
print('total trainable params {}'.format(sum(p.numel() for p in model.parameters() if p.requires_grad)))

optimizer = optim.Adam([p for p in model.parameters() if p.requires_grad],
                       lr=hparams.initial_learning_rate)

checkpoint_path= "/kaggle/working/Wav2Lip/checkpoints/wav2lip_gan.pth"

checkpoint_path= "/kaggle/input/checkpoint-wav/checkpoint_step000279000.pth"

model = load_checkpoint(checkpoint_path, model, optimizer, reset_optimizer=False)

latest_checkpoint_path ="/kaggle/input/wav2lip-expertsync-lombarddata/expert_checkpoints/checkpoint_step000100000.pth"

try:
    if checkpoint_path is not None:
        load_checkpoint(latest_checkpoint_path, syncnet, None, reset_optimizer=True, overwrite_global_states=False)
except:

load_checkpoint("/kaggle/input/wav2lip24epoch/expert_checkpoints/checkpoint_step000060000.pth", syncnet, None, reset_optimizer=True, overwrite_global_states=False)

if not os.path.exists(checkpoint_dir):
    os.mkdir(checkpoint_dir)

train(device, model, train_data_loader, test_data_loader, optimizer,
      checkpoint_dir=checkpoint_dir,
      checkpoint_interval=hparams.checkpoint_interval,
```

```
    nepochs=11)
```

24. Inference code

```python
from os import listdir, path
import numpy as np
import scipy, cv2, os, sys, argparse, audio
import json, subprocess, random, string
from tqdm import tqdm
from glob import glob
import torch, face_detection
from models import Wav2Lip
import platform
import audio

checkpoint_path="/kaggle/input/wav2lip-syncexpertdataset/Wav2Lip/checkpoints/wav2lip_gan.pth"
face="/kaggle/input/girlface/Image.jpg"
speech= "/kaggle/input/girl-audio/11.wav"
resize_factor=1
crop=[0,-1,0,-1]
fps=25
static=False
```

```

**25. Video processing**

```python
if not os.path.isfile(face):
    raise ValueError('--face argument must be a valid path to video/image file')
else:
    video_stream = cv2.VideoCapture(face)
    fps = video_stream.get(cv2.CAP_PROP_FPS)
    print('Reading video frames...')
    full_frames = []
    while 1:
        still_reading, frame = video_stream.read()
        if not still_reading:
            video_stream.release()
            break
        if resize_factor > 1:
            frame = cv2.resize(frame, (frame.shape[1]//resize_factor, frame.shape[0]//resize_factor))
        y1, y2, x1, x2 = crop
        if x2 == -1: x2 = frame.shape[1]
        if y2 == -1: y2 = frame.shape[0]
        frame = frame[y1:y2, x1:x2]
```
```

```
        full_frames.append(frame)
print("Number of frames available for inference: "+str(len(full_frames)))
```

26. Audio processing

```python
if not speech.endswith('.wav'):
    print('Extracting raw audio...')
    command = 'ffmpeg -y -i {} -strict -2 {}'.format(speech, 'temp/temp.wav')
    subprocess.call(command, shell=True)
    speech = 'temp/temp.wav'
wav = audio.load_wav(speech, 16000)
mel = audio.melspectrogram(wav)
print(mel.shape)
```

27. Mel chunk processing

```python
wav2lip_batch_size=128
mel_step_size=16
mel_chunks = []
mel_idx_multiplier = 80./fps
```

```
i = 0
while 1:
    start_idx = int(i * mel_idx_multiplier)
    if start_idx + mel_step_size > len(mel[0]):
        mel_chunks.append(mel[:, len(mel[0]) - mel_step_size:])
        break
    mel_chunks.append(mel[:, start_idx : start_idx + mel_step_size])
    i += 1
print("Length of mel chunks: {}".format(len(mel_chunks)))
full_frames = full_frames[:len(mel_chunks)]
batch_size = wav2lip_batch_size
```

28. Face detection and processing

```python
img_size = 96
pads=[0,20,0,0]
nosmooth=False
face_det_batch_size=16

def get_smoothened_boxes(boxes, T):
    for i in range(len(boxes)):
        if i + T > len(boxes):
            window = boxes[len(boxes) - T:]
```

```
        else:
            window = boxes[i : i + T]
        boxes[i] = np.mean(window, axis=0)
    return boxes


def face_detect(images):
    detector =
face_detection.FaceAlignment(face_detection.LandmarksType._2D,
                                        flip_input=False, device=device)
    batch_size = face_det_batch_size
    while 1:
        predictions = []
        try:
            for i…..
```

29. Video Generation with audio

Download command = 'ffmpeg -y -i {} -i {} -strict -2 -q:v 1 {}'.format(speech, 'temp /result.mp4', outfile)

• Purpose: Combines generated video (result.mp4) with input audio using FFmpeg. • Outputs the final lip-synced video (result.mp4).

30. Evaluation Metrics (LSE-C and LSE_D)

Download !git clone https://github.com/joonson/syncnet_python.git !python calculate_scores_real_videos.py --videofile generated.mp4

31. FID Score Calculation

Download !python -m pytorch_fid original_frames/ generated_frames/

Key Workflow Summary:

1. Setup: Install dependencies, clone repo, create directories.

2. Data Prep: Split dataset into train/val/test.

3. Model Training: o Train SyncNet for lip-sync evaluation. o Train Wav2Lip (generator + discriminator) for lip synthesis.

4. Inference: Generate lip-synced videos from input audio/video. 5. Evaluation: Compute LSE and FID scores.